

2. Intro to Class

What is Class

- Class is just a **blueprint**

Basic Structure

C++

Classes syntax

```
1 class {  
2  
3 }; // keep in mind about the classes
```

What is a constructor?

- A constructor is a function that helps us in initializing a object with some default values

Some Properties

- The **name** is same as the **class** name
- There is no **return** type
- Every class comes with a default constructor

Constructor

C++

Constructors

```
1 class Medhansh{  
2     public:  
3         Medhansh(){  
4             //constructor  
5         }  
6 };  
7  
8
```

Types of constructors

- Default constructor

C++

Default constructor

```

1  A() {
2  x = 0;
3  }

```

- Parameterized constructor

C++ Parameterized constructor

```

1  A(int a) {
2  x = a;
3  }

```

- Copy constructor

C++ Copy constructor

```

1  A(const A& obj) {
2  x = obj.x;
3  }

```

- **copying** the attribute of one object to another

- Move constructor (C++11+)

C++ Move constructor

```

1  A(A&& obj) {
2  x = obj.x;
3  obj.x = 0;
4  }

```

&& -> rvalue reference

& -> lvalue reference

```

1  ~~~

```

This keyword

- It is the pointer to the current object

Names

In the above program thing like `name, age` these are the **attributes** while `sayMyName` is a **method**, **behaviors** of the object

Destructor

- to deallocate the memory used by the object

C++

Destructor

```
1  class students{
2      ~student(){
3          //deallocate the memory object was using
4      }
5  };
```

Some Properties

- The **name** is same as the **class** name
- There is no **return** type
- Every class comes with a default destructor

Everything in one program

C++

Class looks like

```
1  class Student{
2      public:
3          string name;
4          int age;
5          int* hell;
6
7      Student(string name,int age){
8          hell = new int; //dynamic memory
9          this->name = name;
10         this->age = age;
11     }
12
13     void sayMyName(){
14         std::cout<<"My name is"<<this->name<<std::endl;
15     }
16
17     ~Student(){
18         delete hell;
19     }
20 };
```

4 Pillars of OOPs

- 4. Inheritance
- 3. Encapsulation
- 5. Polymorphism

- ➡ 6. Abstraction